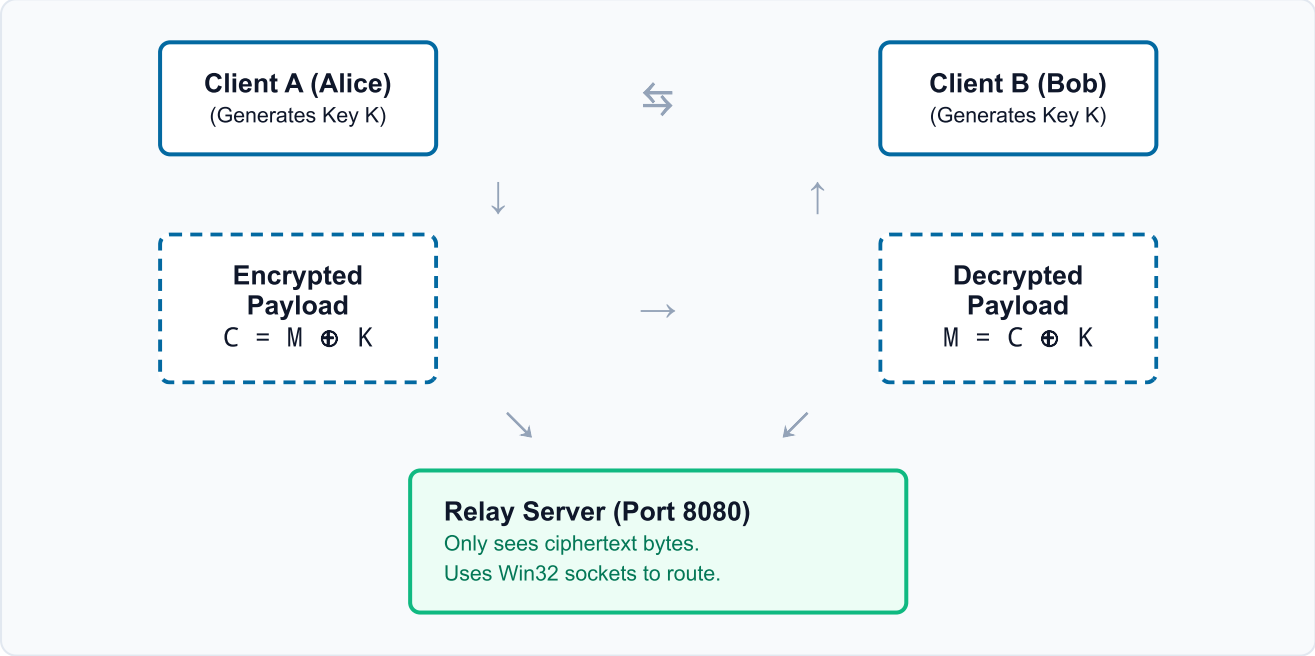
# Architectural Design Document

**Author:** Mohoshin Islam Tonmoy (Roll: BSSE1750)
**Supervisor:** Prof. Mohd. Zulfiquar Hafiz

## 1. System Architecture & Zero-Knowledge Relay

The application is built on a **Zero-Knowledge Client-Server Relay Architecture**. Unlike traditional web-based chat systems that terminate SSL/TLS at the load balancer or central server, this system is designed so that the server acts purely as a dumb byte-router. The server manages socket file descriptors and routes data packets based on username headers but possesses zero capacity to decrypt or inspect the payload.




## 2. Cryptographic Engine: Diffie-Hellman Key Exchange

To establish the End-to-End Encryption (E2EE) session, the clients perform a mathematical handshake using the Diffie-Hellman algorithm over the untrusted server connection. This allows both clients to agree on a shared secret without transmitting the secret itself.

### Mathematical Workflow

Given a publicly known large prime number **p** and a base generator **g**:

1. Client A generates a private secret **a**.
2. Client B generates a private secret **b**.

3. Client A computes Public Key: $\mathbf{A = g^a \bmod p}$ and sends it to B.
4. Client B computes Public Key: $\mathbf{B = g^b \bmod p}$ and sends it to A.

5. Client A computes Shared Secret: $\mathbf{K = B^a \bmod p}$
6. Client B computes Shared Secret: $\mathbf{K = A^b \bmod p}$

Since $(g^b)^a = (g^a)^b$, both clients now possess the identical symmetric key **K**. An attacker (or the Relay Server) observing **A** and **B** cannot easily compute **K** due to the Discrete Logarithm Problem.

#### Stream Cipher

The shared key **K** is fed into a handcrafted bitwise XOR stream cipher. For a plaintext message byte sequence **M**, the ciphertext **C** is generated at the client endpoint as: $C_i = M_i \oplus K_{(i \bmod KeyLen)}$. The exact reverse operation is performed by the receiving client to retrieve the plaintext.

## 3. Multithreaded Concurrency Model

The system bypasses standard high-level async libraries in favor of native `Win32 API` multithreading, ensuring ultra-low latency and minimal memory overhead.

| Component | Thread Strategy | OS Primitive |
|---|---|---|
| **Relay Server** | One Main Acceptor Thread + N Worker Threads (One per connected client socket). Maintains a thread-safe array of active user sockets. | `CreateThread()`, `accept()` |
| **Client: Input Thread** | Dedicated thread blocking on terminal I/O. Captures user commands (`/msg`, `/users`) and plaintext typing without halting incoming messages. | `ReadFile()`, `send()` |
| **Client: Network Thread** | Dedicated background thread continuously listening for incoming data on the Winsock descriptor. Applies ANSI color formatting upon decryption. | `CreateThread()`, `recv()` |

**Memory Management:** To maintain strict execution control, standard C library (`libc`) string abstractions were explicitly avoided. The application manages thread buffers and socket queues using custom pointer-arithmetic routines (e.g., `my_memset`, `my_strcpy`).